

Workshop



# Debugging Agents in Different Environments

Sergiy Dybskiy

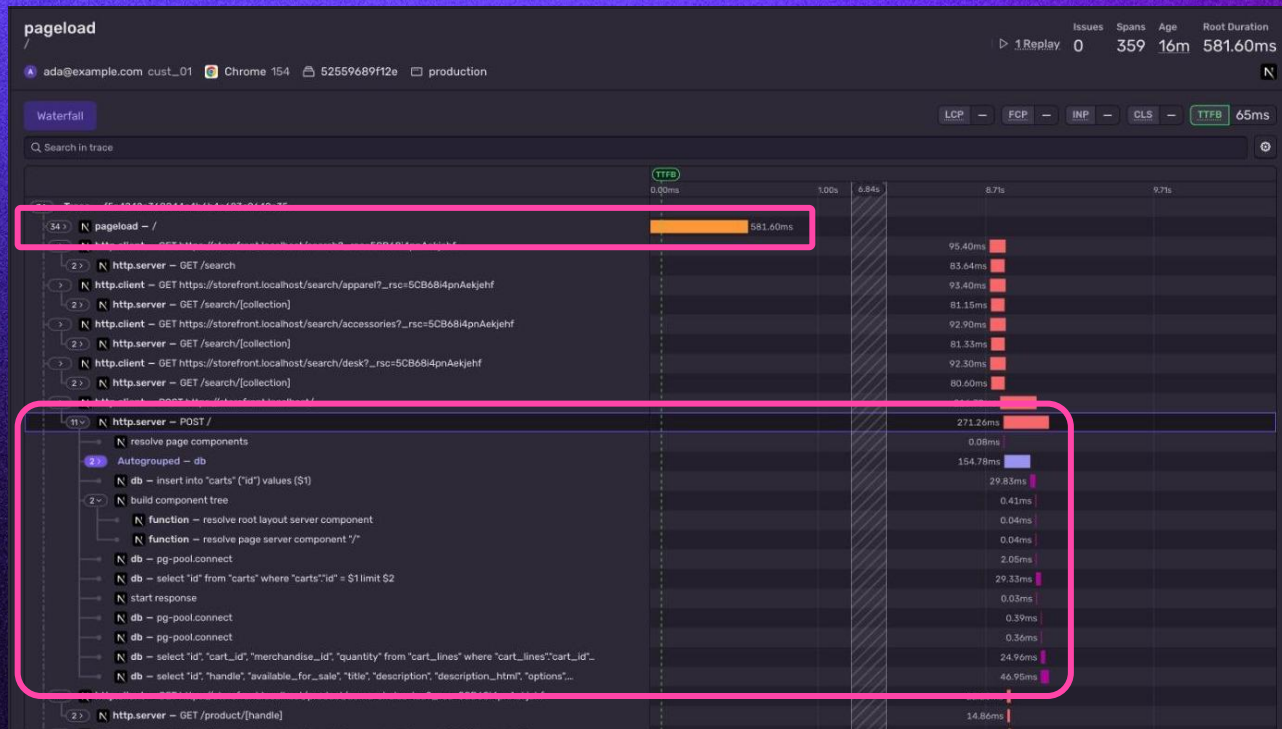
Chief Agent Whisperer

## Agenda

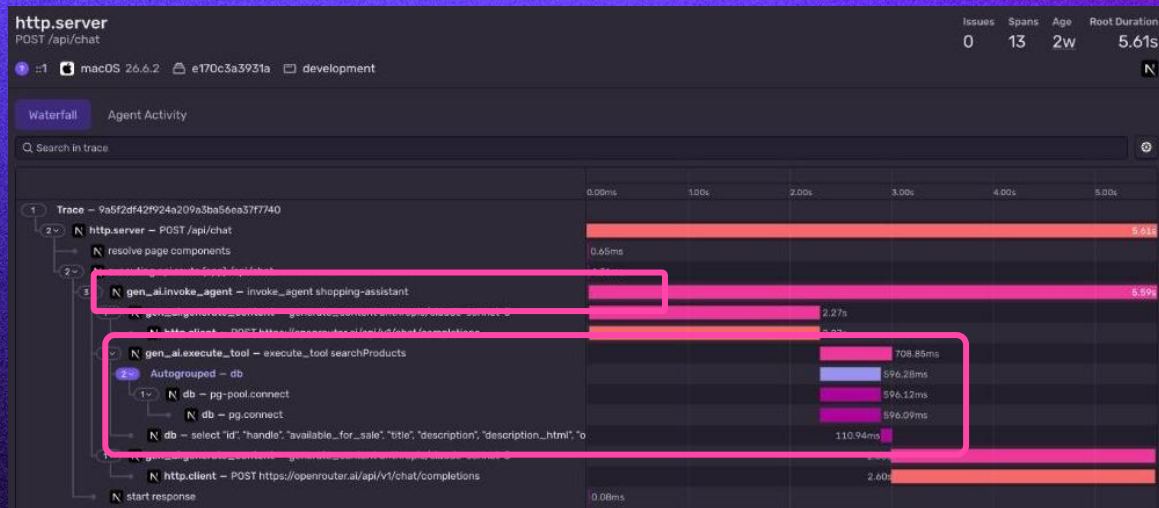
|                              |                                                                                                     |               |
|------------------------------|-----------------------------------------------------------------------------------------------------|---------------|
| <b>What agent tracing is</b> | A request trace, and what an agent adds to it                                                       | <b>5 min</b>  |
| <b>Store assistant</b>       | The customer gets a no. You see why                                                                 | <b>12 min</b> |
| <b>Slack analytics bot</b>   | It works, and it costs more than it should                                                          | <b>10 min</b> |
| <b>Code reviewer in CI</b>   | Set up right. Welcome to Github CI, my friend                                                       | <b>7 min</b>  |
| <b>Alerts and dashboards</b> | Hearing about it before your manager forwards you the anthropic receipt for API usage for the month | <b>9 min</b>  |



# A trace is a picture of one request



# Agents do not take the same path twice



Store assistant

# You are building an online store

It has a chat assistant, because customers ask questions a search box cannot answer.



# Demo



# An agent failure is an application failure

Your evals passed. The agent is fine. It is the rest of your application that it talks to.



# Demo





# How it got instrumented

Install the Sentry plugin for your coding agent: the skills, the Sentry MCP server, the sentry CLI.

Then one sentence: "Add Sentry to this app."

# The agent checks its own work

- Ask the questions you will ask later: who asked, which conversation, which step, how much.
- End the prompt with: send one chat message, then query Sentry and show me the model calls with their token counts.
- It reads Sentry back through the MCP server or the sentry CLI: sentry issue list, sentry trace view.



# Demo





# What the agent trace knew

- Which customer, which conversation.
- That the model chose the refund tool.
- What the tool returned when it failed.
- What the model replied after reading that result.

Slack analytics bot

# Your team has a bot in Slack



# Demo



# It works. What does it cost?



# Demo

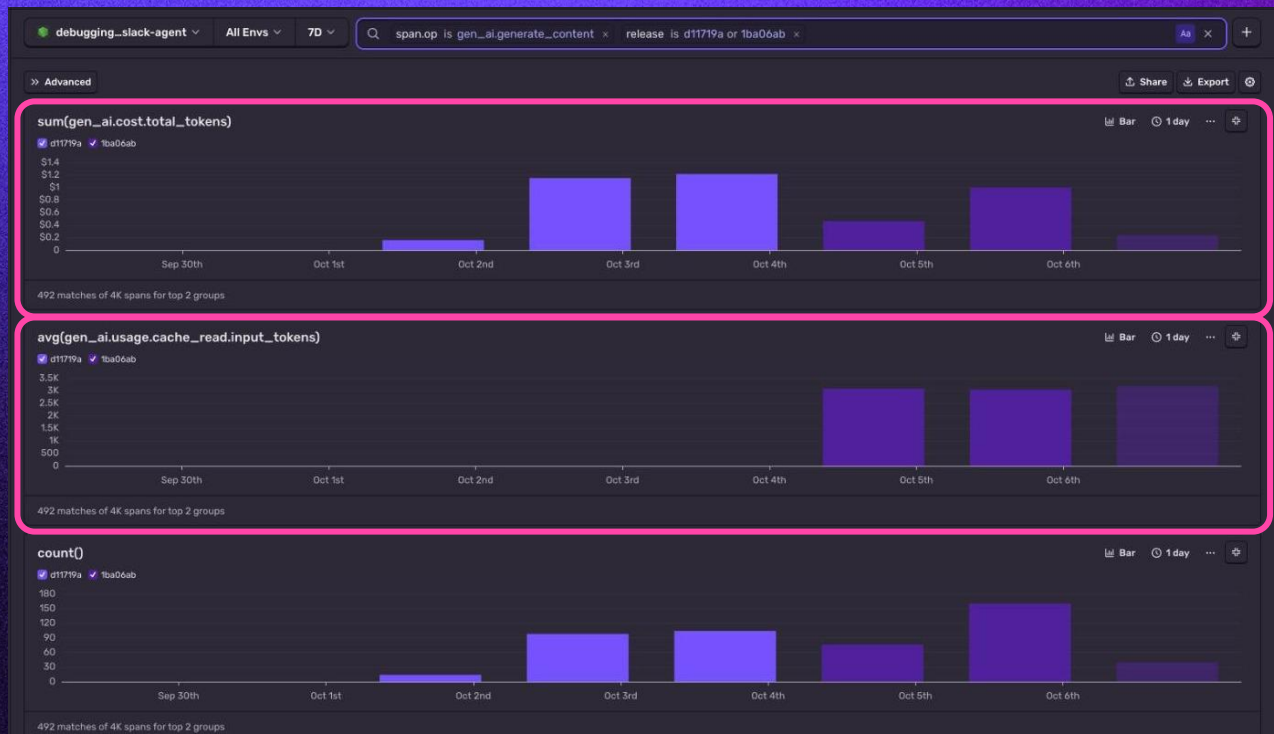


# Same bot, same answers, a smaller bill

The first call of a turn writes the cache. Every call after it reads.  
Nothing failed and no test changed. The trace carried the price.



## Same bot, same answers, by release



Model calls grouped by release over a week. Spend per day on top. Cached tokens per call below. The cached release reads the cache on every call and spends less for more calls.



# Every pull request gets reviewed by an agent

A lead reads the diff, hands one task to a correctness reviewer and one to a style reviewer, runs the tests the diff touches, and posts one comment.

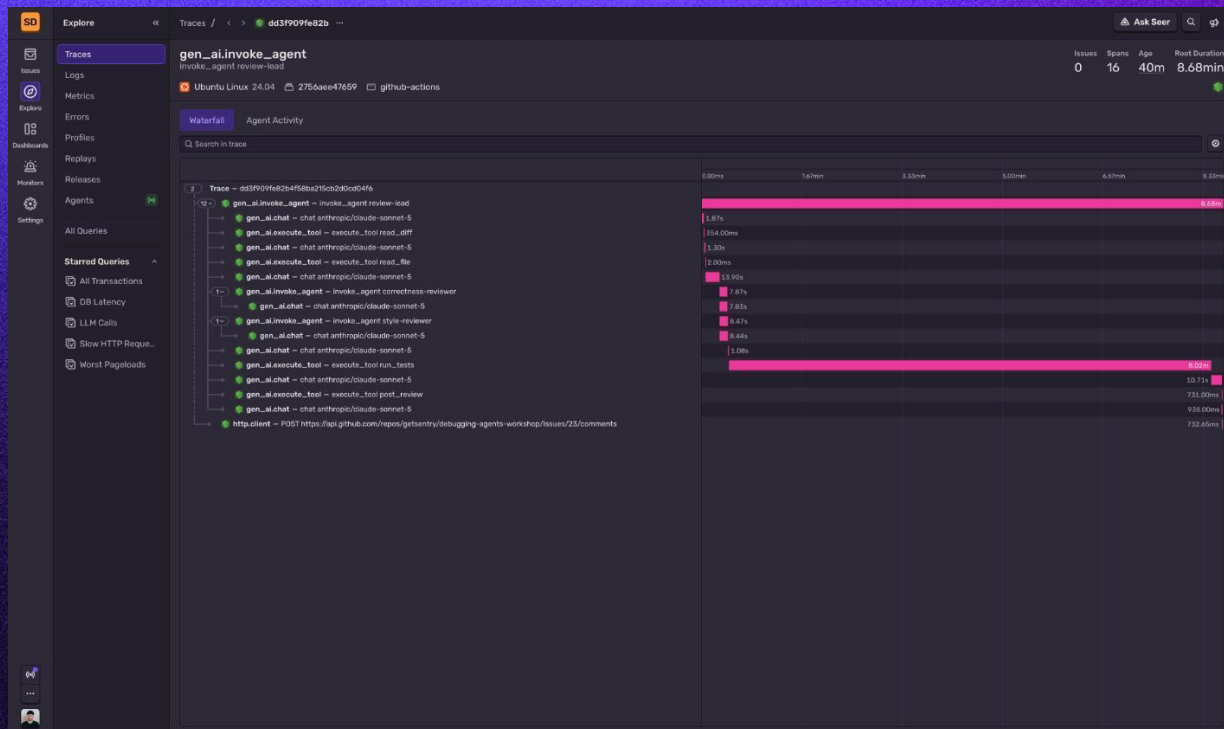




## The tests ran longer than the cache lived

Every call the lead makes reuses a cached prefix: the system prompt, the tools, the conversation so far. By default that cache lives a few minutes.

# One trace



# One line

PI\_CACHE\_RETENTION: long in the workflow. The cache lives an hour.



# What do you alert on when you have no errors

A number on a step, over time.



# Demo



# One dashboard per question

## Slack bot

Who asks, and what one question costs.

## Store

The most expensive customer conversations, and which tool fails.

## CI reviewer

Cost per pull request, and the cache hit rate.



# Demo





# Three ways to make one

- Ask Seer in Sentry, in a sentence.
- The Sentry CLI: one command per dashboard, one per widget.
- The API, from a script in the repo, so the dashboard ships with the agent.

# When it fires, something can fix it

The monitor posts to #sentry-alerts. Reply @sentry in the thread. Seer reads the issue, finds the cause, opens the pull request.



# Take these home

## Agents throw

A failed tool is an application failure. Make it an issue with the conversation attached.

## Cache rules everything around me

You turn it on. The trace shows whether it is working.

## Observe the system

The agent runs alongside your application, database, CI

Watch all of it.



# Questions



[github.com/getsentry/debugging-agents-workshop](https://github.com/getsentry/debugging-agents-workshop)

